

Application: Application is a software which is used to solve the real world problems.
ex: Amazon flipkart, phone pay, Gpay for financial problems.

Types of Appn:

1) standalone: It is a desktop application which does not require browser (or) internet to run. It is already installed in desktop and we can use it.
ex: calculator, alarm

2) web Application: It is a network (or) software that runs on a server and is accessed through a web browser.

ex: Gmail, youtube and Facebook.

Programming: ex: addition, subtraction etc --

Programming is a process of giving instructions to the computer to solve problems (or) task.

Programming language:

It is a formal set of rules and syntax used to write code or programs.

Java

Java is a high level object oriented, platform independent programming language. Java is used to help the developers to build the web, desktop, mobile and enterprise applications.

Features:

1) high level

Java is a high-level programming language because its syntax is easy for humans to read and write.

2) platform independent

Java is platform independent because java code run on any operating system that has a JVM.

3) portable (flexible)

Java is portable because its programs produce the same results on different system. we can transfer our java code one system to another system and it still work correctly.

4) WORA (Write Once Run Anywhere)

write the java program once and run it anywhere without changes code.

* This is the java biggest advantage.

5) Architecture neutral

Java is a architecture neutral because its bytecode is not dependent on any specific processor (or) architecture. It can run on any other devices.

6) object oriented:

Java is based on classes and objects, supporting encapsulation, inheritance, poly, and abstr

How Java Execution works

First code given to Java compiler which converts (HLL) to (JLL) and this JLL is given to JVM to convert (JLL) to (NLL) and machine will process the instruction and prints the output.

Syntax

```
class main {
```

```
    public static void main (String[] args) {  
        System.out.println ("Hello Koderest");
```

```
    }
```

```
} o/p: Hello Koderest.
```

public,
private

Print (vs) println

Print(): Print is a method which is used to print the value and stay in same line.

Ex: S.O. Print ("Hello Jaggu");

O/p: Hello Jaggu

S.O. Print ("Hello");

S.O. Print ("Jaggu");

O/p: Hello Jaggu —

Println():

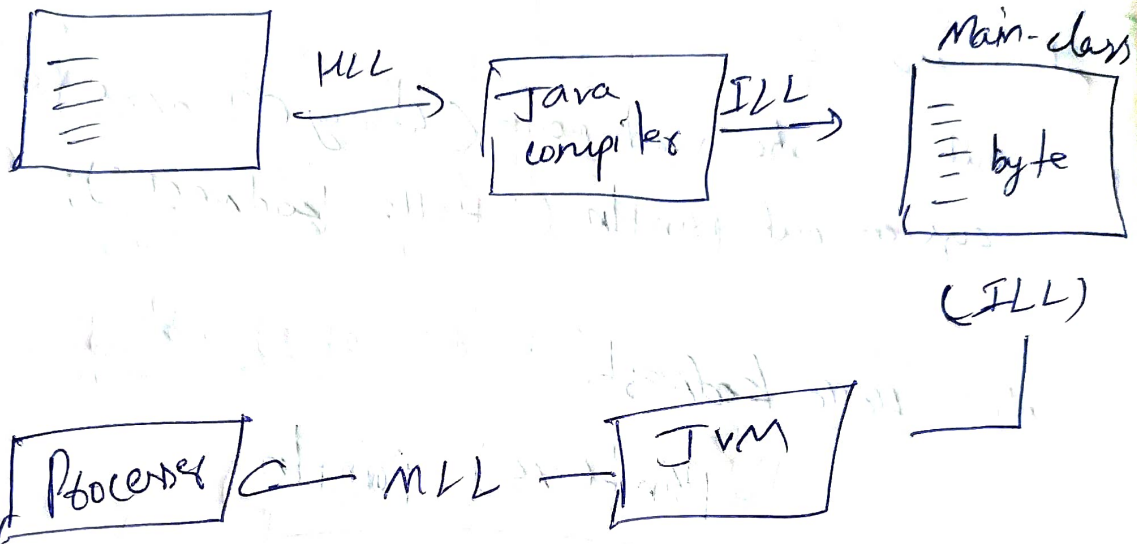
Println is a method which is used to print the value and cursor move to next line.

Ex: `S.D. Println ("Hello");`

o/p: Hello

flow of Java program

main - Java (HLL)



output

Identifiers

Identifiers are the names, which are given to the variables, classes, methods, and packages.

Ex: `int age = 22;` (age is circled and labeled 'Identifiers')

`String name = "Jagge";` (name is circled and labeled 'Identifiers')

`class main { }` (main is circled and labeled 'Identifiers')

This has some set of rules.

→ Keywords should not be used as identifiers.

Ex: class we cannot use as identifier.

- should start with Letters, \$, _
- should not start with numbers.
- should not have spaces in between the words.

Variables:

Variables are names which are given to memory location to store ~~what type of~~ data values ~~variables~~. That can be accessed and modified during the execution of program.

Variables are created by using two methods

- 1) var: using var keyword

```
var age = 22;
```

In this method type inference will happen means java will check the value and come to a conclusion i.e. is a int type.

2) Data type:

By using data type also we can create variables.

```
Ex: int age = 22;  
String name = "Jaggu";
```

Data types

Data type is a classification that specifies that what type of data can be stored in a variable.

Two types

- 1) Primitive : This data types are not considered as objects.
- 2) non-primitive : This data types are considered as objects.

1) byte, short, int, long (To store integer values)
↓ ↓ ↓ ↓
1 byte 2 byte 4 byte 8 bytes

2) float, double (To store real decimal numbers)
↓ ↓
4 byte 8 byte

3) boolean (True & False)
↓ we can't decide byte size it's depends on JVM.

4) char (store character type)
↓
2 bytes

For calculate range

-2^{n-1} to $2^{n-1}-1$

operators

operators are special symbols in java that performs specific operations on 1, 2, or three operands (values, variables) and produce a result.

Types of operators

1) Arithmetic operator

Arithmetic operators are used to perform mathematical calculations such as: sub, add, mul, div.

2) Relational operator

Relational operators are used to compare two values and return a boolean result.

(==, !=, >, <, >=, <=)

3) Logical operator

Logical operators are used to combine or modify boolean expressions. (i.e. logical AND, logical OR, logical NOT).

|| Logical OR, ! Logical NOT

4) Assignment operators

Assignment operators are used to assign values to variables: (=, +=, -=, *=, /=, %=)

5) Increment and decrement

These operators are used to increase or decrease a variable value by 1. (+, -)

6) Bitwise operators

Bitwise operators perform operations directly on binary bits of integer data.

(&, |, ^, ~, <<, >>, >>>)

7) Ternary operator

This operator is shorthand form of if-else statement, that return one of two values based on a condition.

methods

method:

A method in java is a block of code that performs a specific task. It is executed when it is called. Methods help make programs modular, reusable, and easier to maintain.

Ex:

```
public int add (int a, int b) {
```

```
    return a+b;
```

```
}
```

Types of methods in java

predefined:

methods that are already provided by java libraries.

ex: `println()`.

User defined

method created by the programmer according to the application's requirements.

ex:

```
public class main() {  
    static void greet () {
```

```
        S.O.P ("Jaggu");
```

```
    }
```

```
    public static void main (String[] args) {
```

```
        greet();
```

```
    }
```

① no parameter no return type

`void display ()`

② method with parameters and no return type

`void display (int num)`

③ no parameters with return type

```
int getnumber()
```

④ parameters with return type

```
int add(int a, int b)
```

Instance

A method that belongs to an object of a class.

control constructs

Control constructs are statements that control the flow of execution of a program.

1) if

The if statement is used to execute a block of code only when a specified condition is true.

```
if (condition) {
```

```
}
```

2) if-else

The if-else statement is used to execute one block of code when the condition is true and another block when the condition is false.

```
if (condition) {  
    // code to be executed if condition is true  
}  
else {  
    // code to be executed if condition is false  
}
```

3) if-else-if Ladder

The if-else-if Ladder is used to test multiple conditions one after another. The first true condition's block is executed.

```
if (condition1) {  
    // code to be executed if condition1 is true  
}
```

```
else if (condition2) {  
    // code to be executed if condition2 is true  
}
```

```
else if (condition3) {  
    // code to be executed if condition3 is true  
}
```

```
else {  
    // code to be executed if none of the above conditions are true  
}
```

4) switch

The switch statement is used to select one block of code from multiple options based on the value of an expression.

Iteration constructs (Looping)

Looping constructs are used to execute a block of code repeatedly until a specified condition becomes false.

for loop

The for loop is used when the number of iterations is known in advance.

```
for (initialization; condition; update) {  
    ;  
}
```

while loop

The while loop executes a block of code as long as the condition is true. The condition is checked before executing the loop.

```
while (condition) {  
    ;  
}
```

do-while loop

The do-while loop executes the loop body atleast once because the condition is checked after the condition.

```
do {  
    ;  
}
```

```
while (condition);
```

break:

The break statement is used to terminate the execution of a loop.

ex: `int day = 2;`
`switch (day) {`

case 1:

`system.out.println("Monday");`
`break;`

case 2:

`system.out.println("Tuesday");`
`break;`

case 3:

`system.out.println("Wednesday");`
`break;`

o/p: Tuesday.

Continue

The continue statement in java is used to skip only the current iteration.

ex: `for (int i = 1; i <= 5; i++) {`
 `if (i == 3) {`
 `continue;`
 }
 `s-o-p ("Number: " + i);`
}

o/p: 1
2
4
5

OOP

object-oriented programming is a programming style where we create objects that contain both data (variables) and behaviour (methods).

Real life example

car (object) having two things
Properties (data)

- color
- brand

Behaviours (Action)

- start()

• stop()

- Accelerate()

```
class car {
```

```
    string color;  
    string brand; } Data
```

```
    void start() { } Behaviour
```

```
        S.O.P ("car started");
```

```
    }
```

```
    void stop() { } Behaviour
```

```
        S.O.P ("car stopped");
```

```
}
```

why do we use oop?

we use oop because it make code organized, reusable, secure, and easy to understand. It help us represent real-world objects using class and objects.

Ex A school has 500 students.

Instead of writing the same code 500 times, we create one student class.
create 500 student objects.

```
class student {
```

```
    string name;
```

```
    int age;
```

```
    void study() {
```

```
        S.O.OP("student is studying");
```

```
    }
```

```
}
```

```
class main {
```

```
    p.s.v.m (string[] args) {
```

```
        student s1 = new student();
```

```
        student s2 = new student();
```

```
        student s3 = new student();
```

```
    }
```

```
}
```

class

A class is a blue print (oo) template used to create objects.

Real-life example

suppose a company wants to manufacture phones, before manufacturing they prepare a design, that design is called blueprint.

class student {  This is the class.

string name;

int age;

void study();

S.O.P ("student is studying");

}

}

object:

An object is a real instance of class. It represents a real-world entity that has state (data) and behaviors (method).

Real-life example

car

- class - car (Blueprint)

- objects - BMW, Audi, Suzuki, TATA.

```
class student{
```

```
    string name;
```

```
    int age;
```

```
}
```

```
class main{
```

```
    p.s.v.m(string[] args){
```

```
        student s1 = new student();
```

```
        s1.name = "Jagadeesh";
```

```
        s1.age = 22
```

```
        s.o.p(s1.name);
```

```
        s.o.p(s1.age);
```

```
    }
```

Encapsulation

Encapsulation is the process of removing direct access by using 'private' and provided the controlled access by using setter and getter method.

why do we use?

To protect data

To prevent unauthorized access

To make program more secure.

```
class student {
```

```
    private int age;
```

```
    public void setAge (int age) {
```

```
        this.age = age;
```

```
    }
```

```
    public int getAge () {
```

```
        return age;
```

```
    }
```

```
}
```

```
class  
public main {
```

```
    p.s.v.m (String[] args) {
```

```
        student s = new student();
```

```
        s.setAge (21);
```

```
        System.out.println (s.getAge());
```

```
    }
```

static variable

The static variable is declared by using static. These variables which are common for the all objects and it is belongs to class.

Static Block

static block is a block of code declared with the static keyword. It executes only once when the class is loaded into memory, before the main method.

Instance variable:

An instance variable belongs to an object. Every object has its own value.

Instance Block:

An instance block executes every time an object is created, before the constructor.

Constructor

A constructor is a special method that is automatically called when an object is created. It is used to initialize the object's data.

```
class student {
```

```
    student () {
```

```
        s.o.p ( "constructor is called" );
```

```
    }
```

```
    p.s.v.m (String[] args) {
```

```
        student s1 = new student();
```

```
    }
```

Default constructor

If you do not write any constructor in a Java automatically create a one constructor is called the default constructor.

```
class student {
```

```
    p.s.v.d (String [] args) {
```

```
        student s = new student();
```

```
    }
```

Parameterized constructor

A constructor that accepts one or more parameter is called parameterized constructor.

```
class student {
```

```
    String name;
```

```
    int age;
```

```
    student (String n, int a) {
```

```
        name = n;
```

```
        age = a;
```

```
    }
```

```
    void display () {
```

```
        s.o.p (name);
```

```
        s.o.p (age);
```

```
    }
```

```
    p.s.v.d (String [] args) {
```

```
        student s = new
```

```
        student (args[0],
```

```
        Integer.parseInt(args[1]));
```

```
    }
```

No-Argument constructor

A constructor with no parameters that is written by a programmer.

```
class student {
```

```
    student() {
```

```
        s.o.p("No-argument constructor");
```

```
    }
```

```
    p.s.v.d(string C3 args) {
```

```
        student s = new student();
```

```
    }
```

```
}
```

Constructor overloading

Constructor overloading means creating multiple constructors in same class with different set of parameters.

Constructor chaining (this())

It is used to call one constructor from another constructor in the same class.

```
class student {
```

```
    student() {
```

```
        this("Jagadeesh");
```

```
    s.o.p("Default constructor");
```

}

```
Student (String name) {
```

```
    S.O.P ("name: " + name);
```

}

```
P.S.V.D (String[] args) {
```

```
    Student s = new Student();
```

Constructor chaining (super)

It is used to call parent class constructor from child class constructor.

```
class parent {
```

```
    parent () {
```

```
        S.O.P ("parent constructor");
```

}

```
class child extends parent {
```

```
    child () {
```

```
        super();
```

```
        S.O.P ("child constructor");
```

}

```
P.S.V.D (String[] args) {
```

```
    Child ch = new Child();
```

copy constructor

A copy constructor "copies the data from one object to another object in same class."

```
class student {
```

```
    string name;
```

```
    int age;
```

```
    student (string name, int age) {
```

```
        this->name = name;
```

```
        this->age = age;
```

```
    }
```

```
    student (student s) {
```

```
        this->name = s.name;
```

```
        this->age = s.age;
```

```
    }
```

```
    void display() {
```

```
        s.o.p("name:" + name);
```

```
        s.o.p("Age:" + age);
```

```
    }
```

```
    p.s.v.d (string c1 args) {
```

```
        student s1 = new student ("jagadeesh", 21);
```

```
        student s2 = new student (s1);
```

```
        s1.display();
```

```
    } } s2.display();
```

Inheritance

Inheritance is a mechanism in java where one class acquires the properties and behaviours of another class using the extends keywords.

single inheritance

when one child class inherits from one parent class

```
class Animal {  
    void eat() {  
        s.o.p("Animal is eating");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        s.o.p("Dog is barking");  
    }  
}
```

```
class main {
```

```
    p.s.v.m(CString[] args) {
```

```
        Dog d = new Dog();
```

```
        d.eat();
```

```
        d.bark();  
    }
```

multilevel inheritance

when a class inherits from another class, and another class inherits from that child class.

```

class Animal {
    void eat() {
        s.o.p("eating");
    }
}

```

```

class Dog extends Animal {
    void bark() {
        s.o.p("Barking");
    }
}

```

```

class Puppy extends Dog {
    void play() {
        s.o.p("playing");
    }
}

```

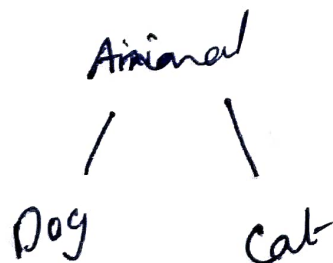
```

class main {
    p.s.v.m(String C args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.play();
    }
}

```

Hieroschical inheritance

when multiple child classes inherits from the same class. (parent).



```

class Animal {
    void eat () {
        S.O-P ("eating");
    }

```

```

class Dog extends Animal {
    void bark () {
        S.O-P ("Barking");
    }

```

```

class Cat extends Animal {
    void meow () {
        S.O-P ("meowing");
    }

```

```

class main {

```

```

    P-S-V-M (storing C) args ) {

```

```

        Dog d = new Dog();

```

```

        d.eat();

```

```

        d.bark();

```

```

        Cat c = new Cat();

```

```

        c.eat();

```

```

        c.meow();

```

```

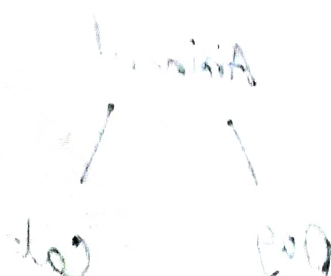
    }

```

```

}

```



multiple inheritance

when one child class inherits from multiple parent classes.

```
interface A {
```

```
    void show();
```

```
}
```

```
interface B {
```

```
    void display();
```

```
}
```

```
class C implements A, B {
```

```
    public void show() {
```

```
        S.o.p("A method");
```

```
    }
```

```
    public void display() {
```

```
        S.o.p("B method");
```

```
    }
```

```
class main {
```

```
    p.s.v.d C obj = new C();
```

```
    C obj = new C();
```

```
    obj.show();
```

```
    obj.display();
```

```
}
```

```
}
```

super() keyword

Super is a keyword in java used to refer to immediate parent class object. It is used to access parent variable, call parent class method, call parent class constructor.

```
class parent {
```

```
    int x = 10;
```

```
}
```

```
class child extends parent {
```

```
    void display() {
```

```
        S.O.P(x);
```

```
        S.O.P(super.x); — call the Access parent variable.
```

```
}
```

```
P.S.V. m(String args) {
```

```
    child c = new child();
```

```
    c.display();
```

```
}
```

```
}
```

o/p: 20
10 —

call parent method

```
class parent {
```

```
void show() {
```

```
    s.o.p("parent method");
```

```
}
```

```
}
```

```
class child extends parent {
```

```
void show() {
```

```
    s.o.p("child method");
```

```
}
```

```
void display() {
```

```
    show();
```

```
    super.show(); // call parent method
```

```
}
```

```
p.s. m. (String args) {
```

```
    child c = new child();
```

```
    c.display();
```

```
}
```

```
}
```

op: child method

Parent method

Final keyword():

The final keyword is used in java to create constant and restrict the changes. It can be used with variables, method and class.

final variable : It cannot be changed after assigned a value.

```
class student {  
    p.s.v.m(string[] args) {  
        final int age = 21; (we can't modify the  
        s.o.p(age);           age because it is  
    }                         final).  
}
```

or

final method()

final method cannot be overridden by a child class.

final class()

A final class cannot be inherited.

Association

It is a relationship between two separate classes where objects of one class are connected with objects of another class.

Ex: A Teacher teaches student, so both can exist independently.

IS - A

Inheritance

HAS - A

Strong HAS - A

Composition

Weak HAS - A

Aggregation

Aggregation

Aggregation is a weak has a relationship between two classes, where child objects can exist independently of the parent object.

Ex: Department has a teacher, if department is removed, the teacher can still exist.

Composition

Composition is a strong has a relationship where child objects depends on the parent object for its existence. Ex: House has a room. The room is considered part of the house.

weak

Has-a

child object can exist
independently

child can be shared with
other objects.

parent doesn't control the
child life cycle.

strong

Has-a

child object depends
parent.

child is owned by
one parent.

parent controls the
child life cycle.

polymorphism

polymorphism is an oop concept in java that allows a method or object to take multiple forms depending on the situation.

Method overloading (compile time)

Method overloading is a feature in java where multiple methods have same name but different parameters within the same class.

class calculator {

void add(int a, int b) {

s.o.p("sum of 2 integers = " + (a+b));

}

```
void add(int a, int b, int c) {
```

```
    s.o.p("Sum of 3 integers = " + (a+b+c));
```

```
}
```

```
} class MethodOverloading {
```

```
    p.s.v.m (String[] args) {
```

```
        calculator obj = new calculator();
```

```
        obj.add(10, 20);
```

```
        obj.add(10, 20, 30);
```

```
}
```

```
}
```

o/p:

sum of 2 integers = 30

sum of 3 integers = 60

Method overriding

Method overriding is a feature in java where a child class provides its own implementation of a method that is already defined in its parent class with same method.

```
class Animal {
```

```
    void sound() {
```

```
        s.o.p("Animal makes a sound");
```

```
}
```

```

class Dog extends Animal {
    @Override
    void sound() {
        s.o.p("Dog barks");
    }
}

```

```

class main {
    p.s.v.m (String[] args) {
        Dog d = new Dog();
        d.sound();
        d.sound();
    }
}

```

O/P: Dog barks

Up casting

upcasting is the process of converting a child class object into a parent class reference.

Down casting

Down casting is the process of converting parent class reference ^{back} into a child class reference.

instance of

It is used to check whether an object belongs to a particular class or not.

Abstraction

Abstraction is the process of hiding unnecessary details and showing only the important information to the user. Abstraction can be achieved using abstract classes and interfaces.

Important rules

- 1) we cannot create an object of an "abstract" class.
- 2) Abstract class can have abstract methods.
- 3) Abstract class can have normal methods.
- 4) child class must implement abstract methods.
- 5) Abstract ^{class} can have constructors.

differs b/w abstract class and normal class?

An abstract class cannot be instantiated and can contain abstract methods, whereas a normal class can be instantiated and cannot contain abstract methods.

Interface

An interface is a blueprint of a class that is used to achieve abstraction and multiple inheritance

in java.

It is given below

Abstract Interface

declared using abstract

declared using interface

classes uses extends

class uses implements

can have abstract +
normal methods

used to define rules

can have constructors

cannot have constructors

It have instance
variables.

variables are public

static final by default

A class can extend only
one class

A class can implement
multiple interfaces

Default method interface

A default method is a method inside an interface that has a body and uses a default keyword.

Static method interface

Static methods belong to the interface itself and are called using the interface name.

Functional interface

A functional interface is an interface that contains exactly one abstract method. It is used with lambda expressions.

Lambda expression

A lambda expression is a short way of writing the implementation of a functional interface

= `() -> {`

`};`